

Video Transcripts

Module 2: Artificial Intelligence Technology Fundamentals: Machine Learning

Video 1: Overview and Taxonomy (07:23)

Machine learning, and especially deep learning, has received an enormous amount of attention in the last couple years. In fact, many people have the impression that deep learning and deep neural networks are synonymous with machine learning. But really, there is a very large variety of machine learning methods and algorithms, and a major challenge for implementing AI products is to be aware of this range of options and methods, and to match the problem at hand, with the appropriate method. So, our goal in this module is to expose you to a range of machine learning methods and provide you some basic insight into how they work.

Let's take a look at a rough taxonomy, or characterization, of machine learning methods. First, we note that this taxonomy, even though fairly complex, is only partial, and indeed, there are even overlaps between branches of this tree. However, this taxonomy allows us to highlight some of the top-level characteristics of different machine learning methods.

One of the most important characteristics relates to how these models are trained or learn from available data. This corresponds to the very top-level of our taxonomy. The most common scenario is supervised learning, where we have available to us some training data, that has both inputs or features, x , and some labels or output values, y , corresponding to each input. For example, we might have lots of images of cats and dogs, each having a label indicating it is, indeed, a cat or a dog.

A second important scenario is unsupervised learning. In this case, we only have inputs or features, but no corresponding labels or outputs. In this case, we might employ machine learning methods that seek to discover or model some kind of structure, within that data itself. For example, we might have just a big pile of images, and what we'd like to do, is create 10 different piles or clusters of images, that seem most similar to each other, within those clusters or most different between different clusters.

The third key scenario is what I will broadly call, semi-supervised learning. In this case, we might only have a subset of data, with both inputs and known outputs, and the machine learning task might involve searching the space, to find some optimum. Perhaps, where the machine learning model gets to pick, what new data point should be acquired, in order to best explore the space of possibilities.

A very interesting case is reinforcement learning, where the machine learning method may train over one or many experiences, whereby the algorithm sometimes explores, and sometimes seeks to maximize some reward, to learn how to take good actions, leading to good reward in that environment. AlphaGo is a famous example, of a reinforcement learning algorithm, that teaches itself, how to play the game of Go, even better than humans are able to.

A second important characteristic relates to the nature of the models that are constructed by the machine learning algorithm. Parametric models are what we're most used to, for most engineering and scientific experiences. For example, in fitting a line or curve, to some x data. The key idea in parametric models, is that we typically know the model structure, and indeed, the model complexity, in terms of the number of model coefficients, and those don't depend on the number of training samples.

I should note that parametric methods, are often accompanied by additional mechanisms, like regularization or additional hyperparameters, that seek to control the complexity of the model, but the model form typically does not depend on the number of training data points. Non-parametric methods, on the other hand, are less familiar to us. These are models where the model complexity depends on the number, or nature, of the training samples.

A very simple example, might just be a table, reporting all previously observed xy pairs, and when you get a new x value, you just guess the y value, for the nearest previously seen x point. One can see that the complexity, or the size of the table, depends on the number of training samples, rather than having a fixed size or complexity. Note that this example may seem trivial, but in fact, it is the basis for a formal machine learning method, called nearest neighbors.

The final characteristic, we highlighted in this taxonomy, is the nature of the input/output relationship, and the implied, or explicit probability model that those relationships capture. First, are discriminative methods, that directly learn the probability of output y , given an input x . Maybe, it's the probability that an image is a cat versus being a dog. In some sense, this is a forward model. Given an input, predict the probability of an output y .

In contrast, are generative methods. These seek to model these as the joint probability of x and y , or in some cases, the reverse probability. Given an output y , what is the probability that different x values are associated with that y ? In some AI products, this might be incredibly useful. If we observe an anomaly or failure in a manufacturing line, we might want to model the probability, that different equipment settings, x , might have been responsible for that failure.

Some machine learning methods combine these forward and reverse predictions. A very interesting set of recent methods are GANs or generative adversarial networks, that combine these features, and are able to do some really impressive things, like generate very realistic-looking faces or images of people's faces that, in fact, are fake.

Now that we've armed you with these high-level characteristics, in a sense of the range of machine learning methods, we will touch on a number of methods, and core concepts, deeper down in our taxonomy. Our goal will be to build some intuition about the key ideas behind these methods, so, you can start to understand the benefits and limitations of them, as you later think about what machine learning method might match up best to a particular problem or an AI product that you might envision.

Video 2: Applying AI to Identify Genetic Predisposition to Diseases (07:07)

It is generally recognized that many diseases have a genetic origin or at least, that each of us has a generic predisposition to some diseases over others. Knowing what your particular weak point is may have developed a tailored approach to minimize negative consequences for you. As an example, take lung cancer, which is the most deadly of all cancers today. 80% of the cases are linked to avoidable causes like smoking, but at least 8% of the cases of lung cancer are linked to genetic causes. And within these 8%, it kills by surprise, because there is little warning unless very expensive onset diagnosis are performed which are beyond current reach of most health care systems. Wouldn't it be nice if we could tell who has this early predisposition?

Early detection of lung cancer vastly increases the probability of survival. The treatment is both more effective and much cheaper. Wouldn't it be great if you had the option of knowing if you were in this 8% risk group and were able to take action? Professor Manolis Kellis has a deep understanding of the evolution of living organisms and has demonstrated how AI can help. It's a lot of fun to talk to him, and he, in fact, claims that competitive evolution has been replaced by what he calls collaboration, as it will be clear by the end of the segment. Manolis Kellis is part of the computational biology group at MIT. By computationally integrating functional and

comparative genomic datasets, he and his team aim to advance the understanding of the human genome. His main goal is to comprehend the effects of genetic variations in human diseases such as lung cancer, diabetes, Alzheimer's, schizophrenia or obesity.

To do so, his team uses linear regression-based models, tree-based models, and support vector machines among other tools. This is one of the cases where multiple tools of machine learning are being used together, so it's not just one type of sort of AI tool but many that work together. And one of the projects of Manolis Kellis group is the development of gencode. Gencode aims to be an encyclopedia of genes and genetic variants. Through a combination of computational analysis, manual annotation, and experimental validation, it aims to eventually identify all genetic features of the human and mouse genome, among others. For accurate use in genomic medicine and genomic biology, they are slowly building a growing collection of both human and mouse genes for research and clinical testing. It aims to ultimately contain all gene characteristics of the entire human genome with high accuracy, systematically bridging the gap, separating common from rare gene variants.

Using machine learning, Professor Kellis' group developed computational methods to determine patterns of sameness and the similarity between closely related genomes. The objective is to develop methods for understanding genomes with a view to applying them to the human genome in medical applications. Here is where the vision of Professor Kellis becomes extraordinary. He has demonstrated that you can use machine learning to tie the genome of different cells with the root cause of the mechanisms producing a given unwanted behavior like, obesity. Using himself, that is Professor Manolis Kellis himself, as an example, his research has found a new combination of genes that interact, so that certain cells can burn excess fat and instead store it generating obesity. In experiments with mice, he has shown how his own genomic variations also prevent a mouse from burning fat.

By understanding the exact mechanism that was miss functioning, they were able to reverse this behavior in mice to the point where an obese mouse with his genomic characteristic, that is with Professor Manolis Kellis' genomic characteristic, ended up being completely normal and never obese again. It was fascinating. He uses the word "pathways" to refer to the way in which many genes interact to produce a syndrome like obesity. He contends that each person is different because of evolution and that one needs to break the traditional silos in medicine. He has a grand vision that using machine learning, we will learn the wrong turns of evolution to the point of reversing it in the case of diseases. That's why he says, he wants to put an end to evolution on Darwin's described way. So, his idea is that by sharing all these genes and pathways and the mechanisms we will learn the wrong turns of evolution collaboratively to the point where we will

end the "bad turns" of evolution, is quite an ambition goal. But hey, you know, Professor Kellis is at MIT.

Video 3: Linear Classifiers (03:14)

We'll start with one of the most basic and important machine learning methods, linear classifiers, trained using supervised learning. We'll assume we have some multi-dimensional input data, some feature vector x . For the sake of an example, we'll picture a two-dimensional feature space, with variables $x^{(1)}$, the horizontal axis, and $x^{(2)}$, the vertical axis.

For each of our data points, we also have an associated label indicating the class, the point belongs to. For now, we'll just consider binary classification; a point is either in the positive (plus) class or the negative (negative sign) class. A linear classifier has a particular model form. It's given by a linear separator, $\theta^T x + \theta_0 = 0$, where the θ vector defines the positive side of that linear separator. For example, the line pictured here, is given by θ being the vector minus 2, 3 with $\theta_0 = 6$.

Once we have a linear classifier, for any new data point, we can predict the positive class if $\theta^T x + \theta_0$ is greater than zero; otherwise we predict the negative class. Training or learning is done by some algorithm, that uses some set of training data to find the coefficients to do a good job, in both the training data, but also importantly, in new data, that we will encounter when we put the classifier into use.

We'll return to training in a moment, but first, we note that even what appears to be a very simple model, a linear classifier, is able to produce models that can be actually quite complex. First, we may have more features than just x_1 and x_2 . For example, if we had three features, the line we saw in 2D becomes a plane in 3D. Indeed, we might have dozens or hundreds of features for each data point; in which case, the separator becomes a hyperplane.

Even more importantly, we can also create new features, out of those that are provided to us directly, using feature transformations that are calculated as functions of x . For example, we might create polynomial expansions of the original features, in which case, the decision boundaries can become nonlinear in the feature space.

Indeed, the resulting model might have the capacity to produce very complicated decision boundaries, in an attempt to fit the particular training data it is provided. In the next segment, we'll consider the issue of training, and potentially, overtraining or overfitting in more detail.

Video 4: Train, Validate, Test (01:53)

One of the key questions we face with any trained model, or in comparing alternative models, is how good is our model? For example, we might have the red model, h_1 , that has zero errors on the training data shown here. Alternatively, we might consider the blue, h_2 model, which is much simpler, but it has two classification errors in the training set. Which model is better?

Well, the answer is, it depends. In truth, how good a model is, really depends on how it will perform once put into use. We might well worry, that the h_1 model is overfitting to the details of the particular training set, but will not generalize well to data we haven't seen yet.

So, it may be, in this case that the h_2 model has worse training error, but actually has much better test or production error, than h_1 . For most machine learning models, a standard practice is often used to help estimate or evaluate how good your model is, and how good your training algorithm is.

Starting with the data at hand, we pull out some subset, the test data, and never use it during training, and only use it as a stand-in to evaluate error on unseen data. The other part of the data, we can also separate, into training data and validation data. Another approach is regularization, which we'll return to, in a future segment.

Video 5: Decision Trees (05:40)

So far, we've seen linear classifiers that are parametric, specifically the classifier is a linear model, with parameters θ and θ_0 . Next, we'll consider another very often used machine learning model, a decision tree. A decision tree can be pictured in a couple of ways.

The first, corresponds to the name. We have a tree structure that has some condition, or split on one feature at a time, starting from the top of the tree. To make an inference, for a new example, with features f , we simply traverse down the tree, taking the less or true branch to the left, if that is the feature, if that feature is less than or equal to some split value, or we take the greater than, the false branch to the right, otherwise. We continue until we reach a leaf node where the label for the class is given.

The second way we can picture a decision tree, is being a successive binary partition of the feature space where we associate each leaf partition, with the classification for that region. In both of these views, we can see that the decision tree is non-parametric, in that the complexity

of the tree, depends on the complexity of the model, training data, not on some a priori fixed number of model parameters. That makes tree models attractive for easily adapting during supervised training to the available data.

Most algorithms for learning a decision tree use training data to recursively construct the tree. Given the data in some region of the tree, one finds a best single feature, $f_{\text{sub } i}$, and corresponding value to split on, and we continue this recursively on each branch until some stopping criteria is reached, and a class label is associated with that leaf node.

The definition of best split has some flexibility. It's often associated with how pure the membership of data points are in each side of the split, e.g., with respect to a Gini index, entropy, or a misclassification error. One advantage of a decision tree is that the model is inherently or easily interpretable. For example, given a decision tree on good versus bad part classification, one can answer a question like, why is this part identified as being bad? And can answer it with, because its f_2 value was less than or equal to 27.5, and its f_1 value was greater than 400.

We can also easily extend the decision tree classifier to multiple classes, and more importantly, we can extend the information in the leaf node, to give a probability of membership in a given class, typically by using the fraction of training samples in the leaf associated with each class.

For example, we may say that the probability of being a bad part in the leaf node shown here is 29 out of 30. But there remains a one in 30 chance, that the part is still good, even with an f_1 and f_2 values leading to that leaf node. We will return to this probabilistic aspect of machine learning classifiers in our next segment. Before we turn to probabilistic models, we want to give some insight into two advanced decision tree methods, that have proven to be very powerful in realistic situations.

One limitation of a single decision tree is that it can be very sensitive to the training data. Both, the structure of the tree and some of its predictions can change greatly, if a data point is removed or changes slightly. One way to overcome this is called bootstrap aggregation or bagging. The idea is pretty simple, to take a random sample of the data and train a tree, then take a different random sample of the training data to create another tree and so on. Then, the overall ensemble tree makes a prediction based on a majority vote or averaging of the predictions, from those multiple parallel trees.

The well-known random forest machine learning model is a kind of ensemble tree, that not only uses bagging, but also randomly limits itself to subsets of the feature choices, so that the random forest becomes more robust across the full set of features in the training data.

A second kind of combined set of decision trees results from what is called boosting. In this case, one decision tree is based on the training data, but then a second decision tree is trained specifically on the errors in training prediction coming from the first tree, to compensate or account for those errors. A boosted decision tree might have several such correcting decision trees in sequence.

Video 6: Bayesian/Probabilistic Models (05:46)

In the case of decision tree classifiers, we noted that it is possible for a tree to provide a probability of an example, being in a class. This highlights that many machine learning approaches are probabilistic in nature. They might do more than just suggest a best guess, for an output label or value, for a given example. They might also seek to provide a probability or confidence associated with that example.

Bayesian methods are a broad family of machine learning approaches, that explicitly model and account for beliefs about models, sometimes, beliefs about outputs, and sometimes beliefs about parameters of the model themselves. Bayesian methods have mechanisms for how these beliefs change or are updated when a new piece of data or information become available. That belief is generally expressed as a probability, for example, a probability density function, or pdf, about the input or outputs or model parameters.

Typically, one has some prior belief, either based on domain knowledge or previous experience or data up to the current time. For example, one may believe or approximate the pdf, for some model parameter, as being normal or Gaussian distributed, with mean μ and variance σ^2 . Suppose now that we have a new observation, y_i . What we would like to do, is generate an updated posterior belief, in the model parameter; that is, an updated pdf of θ given y_i . To obtain that, we make use of another probability relationship, the likelihood that we would have observed y_i , given any possible θ .

The famous Bayes' rule, then tells us how to achieve the update, such that the new pdf for θ , is proportional to the overall pdf of that θ , times the product of the probability of observing y_i , given that θ . Looking at the structure of this Bayesian approach and Bayesian update, we can see some powerful benefits. One is that we have a mechanism to take advantage of prior

experience and knowledge, rather than just throwing all that away and depending only on our training data to create a model. Second, the probability models inherently handle uncertainty. For example, we can inherently deal with noisy measurements in this approach.

Finally, Bayesian methods can blend with adaptive or active learning methods, where we may get to make a choice about the next data point to acquire. For example, we may decide we need to sample a part of the feature space, where our uncertainty about the model is worst, so that we can get the large gain or improvement in our model, when new measurements or data points are expensive.

In this example of a Bayesian update, we start with some prior beliefs about the state of the world. We believe the world is in state or hypothesis H_1 with say 30%, H_2 with 50%, and H_3 with 20% probability. We also have known likelihoods of observing output A , B , or C , given the state of the world. For example, if we're in H_2 , there's a 70% chance we'll observe B , and only a 10% chance we'll observe A . Now, we get a new piece of data. We make an observation. It turns out that we actually observe A . How should we update our belief, given this new evidence? First, conceptually, we see that the joint probability of seeing both H_1 and A , is 0.15, which is substantially higher than the probability of H_2 and A at 0.05, which is also much higher than the probability of being in H_3 and observing A , at just 0.01.

So, Bayes' formula now tells us how to update and get a posterior probability of being in state H_1 , given A as being equal to the probability of H_1 and A divided by the total probability of seeing A , which is 0.15 plus 0.05 plus 0.01 in our example. Thus, our posterior belief, that the state of the world is H_1 has grown; this is supported by the fact that we actually observed A and has grown from 50% to 71.4%. While things get tricky with Bayesian methods, the basic idea is to remember the following: when we get new evidence, we can update our beliefs, giving us a way to incrementally learn, as more data becomes available.

Video 7: Regression and Gradient Descent (04:26)

Up to this point, we have mostly discussed machine learning methods for classification, that is, modeling and predicting the membership of data samples in one or more discrete classes. However, it is also possible for our machine learning models to have continuous outputs.

One case might be the continuous probability of being in a class, but more generally, we can have a wide range of different, continuous outputs. For example, a machine learning model that

predicts the outdoor temperature for the next day, based on sensor signals from the previous day. Such models are generally referred to as regression models, which we will talk about next.

We'll also highlight a powerful mechanism, for incrementally training or learning coefficients of such models, using what's referred to as gradient descent. The set up for learning a regression model can be described as follows. We seek a model h that takes some input x and has some model parameters θ , and predicts an output y , corresponding to the input vector x . Usually, the model has some known structure or form. For example, it might be a linear model, like we saw earlier, with linear classifiers.

Now, y equals $\theta^T x$, plus θ_0 . We then seek to find values of θ , the argument θ , or θ_0 , that minimizes some objective or cost function, $J(\theta)$. A typical objective has both a loss component, such as the loss function L , for errors averaged over our end data points, and a regularization component, R of θ , that expresses some constraints or preferences or penalty, on the model parameters themselves.

There are a variety of algorithms that can be used to find good θ values. That is, to train or learn the model. If the structure of the model is linear in the coefficients, and the loss is a squared error function, this is known as ordinary least-squares regression, and a direct analytic solution is possible. If we add a regularizer, that penalizes for the sum of squared model coefficients, this is known as ridge regression. This can be thought of as preferring models, with small θ coefficients. Other regularizers are possible, such as the sum of absolute values of the coefficients, which would have the effect of seeking simpler models, with fewer number of non-zero model parameters.

A very powerful and general alternative perspective is to employ gradient descent. The idea here is, that we calculate the gradient, ∇J , with respect to the coefficient's θ , and then, on a gradient descent update, we update the model coefficient, by going some increment downhill, following that gradient, that will reduce the cost function J . Here, there is usually some kind of learning rate parameter, such as η , here, to govern how fast or how consistently, we can iteratively reach a good local or global minimum.

An important variant is stochastic gradient descent or SGD, where we randomly pick individual training data points, or batches of training points, and iteratively or sequentially, do the model updates using those subsets. We'll see later that this is the workhorse for training not only things

like simple linear or nonlinear models, but also incredibly complicated models, like deep neural networks.

Video 8: Logistic Regression and SVMs (03:44)

From this regression perspective, we're now ready to revisit classification problems, and frame learning classifiers as optimization problems. This will both give you better intuition behind how regression and gradient descent work, but also, teach you about two very common and well-known machine learning models, logistic regression, and support vector machines, or SVMs. Logistic regression is a powerful, fundamental machine learning method used for classification, but built on a regression framework.

We again have linear models of the form z equals $\theta^T x$ plus θ_0 as a starting point, but now, we add a non-linearity, the logistic function $\sigma(z)$, that squashes z into the range from 0 to 1. This is particularly valuable, because now we can interpret a guess, g , equal to $\sigma(z)$ for any input x , as the probability, that x belongs to the positive class, where we now label positive classes as plus one, and negative class as zero. For training using regression, we need to provide an objective function.

Here, we used the negative log likelihood, as defined here, which penalizes classification errors, when guess g does not match up with the actual class y , for our training sample. We add a regularization term, to push towards small θ values, and then, typically do gradient descent or stochastic gradient descent, to find θ and θ_0 , to minimize this cost function.

We find that this logistic regression, can often do surprisingly well, at finding a good linear classifier, that both trains well, and importantly, generalizes well to unseen data. We will mention one more machine learning classification model, an algorithm, support vector machines, or SVMs.

In this case, we again have a linear model as a starting point, but now our notion of goodness is extended to both, avoid classification errors, and to provide a safety margin when possible, so that the separator is not too close, even to the points that it is correctly classifying. The details involve a different loss function, using what is known as a hinge loss, to capture this goal, but then, we use the same kind of overall objective function, with this hinge loss, instead of the negative log likelihood loss, that we saw with logistic regression.

While SVMs have become less popular recently, they can still provide a powerful method, for creating classifiers that are good or improved, by including this notion of margin. We add a regularization term to push towards small theta values, and then typically do gradient descent or stochastic gradient descent, to find theta and θ_0 , to minimize this cost function. We find that this logistic regression can often do surprisingly well, at finding a good linear classifier, that both trains well, and generalizes well to unseen data.

Video 9: Unsupervised Clustering (04:18)

In addition to supervised learning, where we know both, input feature values, and output or labels, for each sample in our training data, we are often presented with a large volume of data, that are only inputs or only outputs, depending on your perspective.

There is still much to be learned from such data. For example, statistical properties of the sample data, like the mean and variance, are one of the simplest unsupervised models. Other more elaborate ways to describe, or in some cases, compress the available data, are often used.

One common example, drawing from the statistical foundation of machine learning, is principal component analysis, or PCA, which seeks to express the variation in the observed data using a simpler and often reduced dimension approximation to that data.

Another very common machine learning method is clustering, which we will consider next. Clustering is again an unsupervised training method, where we only have feature vector x , for some number of samples. Our goal now is to identify natural groupings of the data, typically where there is substantial similarity between members of the same cluster, and hopefully, good separation between clusters. Once we have created clusters, a common inference to ask might be, to identify what cluster a new point belongs to. Clustering is a good basic method, that is sometimes useful as part of a larger machine learning solution, in combination with other machine learning methods.

For example, there might be a high cost associated with obtaining a label for any given data point, and we have many more data points than we can afford to directly label. Here, we might start by clustering, and then pick only a few representative data points, to directly obtain labels for. And then impute the label for the rest of the points in the cluster, to have the same label.

Let's consider a very popular clustering algorithm, the k-means algorithm, to get a sense of how clustering might work. Here, we start by assuming k different cluster means, maybe randomly

sampled across our feature space. Based on those means, we next assign each data point to the nearest cluster, the nearest cluster mean. Then, we update the cluster means, based on those assigned data points.

We can iterate until some stopping condition is met, such as, when clustering assignment has converged, and points are no longer changing clusters. One challenge in basic k -means clustering, is that one has to assume k , the number of clusters, beforehand. Thus, k is a hyper-parameter of the algorithm, and additional approaches might be needed to try or sweep across different values of k , in order to find one that results in the best clusters, for example, to achieve good separation between clusters.

Other clustering algorithms also exist, again in support of the goal of finding some structure in the data, even without explicit input/output labels. Hierarchical clustering is an example. In this case, one might have a first split of the data, that results in a high-level pair of clusters, and then we recursively divide those splits into smaller clusters. This can give one a sense of groupings of clusters, with similarity or differences identified between them.

Video 10: Semi-Supervised and Reinforcement Learning (04:12)

We have covered key concepts of supervised learning and unsupervised learning. There is a rich and fascinating set of machine learning methods, that sit in between these two extremes, which can be broadly considered, semi-supervised learning.

One key idea, underlying many semi-supervised learning methods, is that the learning algorithm itself may provide guidance on what new data points should be gathered. The first method we'll mention is Bayesian optimization, which builds on the probability model view of the world, we discussed earlier. Because we have probability density functions, or PDFs, telling us our uncertainty about our beliefs, we can look at that uncertainty, and decide where a new data point might best reduce that uncertainty.

For example, we may have a model of an objective or goal function, we're seeking to minimize, that has been built based on just three observations so far, the red points. As we get further away from those known data points, our uncertainty increases, as indicated by the light blue intervals. We might simply pick a new data point that is in a very wide, light blue region, and that indeed would improve our model.

Even better though, is to consider both, the improvement in the model and the probability that such a data point will give us a more optimal objective function. So, in this case, the acquisition function steers us to pick a point in the middle. With subsequent steps, we see that this approaches the overall optimum, with increasing confidence.

A very different, and very important semi-supervised learning method is known as reinforcement learning. In this case, the algorithm, the "learner", is not told directly, what outputs go with what inputs. Instead, the learner has to try different actions and see what works and what doesn't by interaction with the environment. A typical setup is shown here.

First, the learner observes the current state of the world. By the way, this state and observation, can be probabilistic, like in Bayesian models. Based on the current state, s_i , the learner takes an action, a_i , in the environment. And we observe what immediate reward, r_i , occurs. That action also causes the state to change to some new state, s of i plus 1. The goal of the learner is ultimately to learn what actions are best, in order to achieve the best rewards, but not just the best immediate rewards, but rather, the best long-term rewards.

So, the learner needs to again explore this state-action space; build models of rewards, either immediate or ultimate rewards; and improve its knowledge and models, so that it can best exploit those models and achieve some long-term goal.

Amazingly, reinforcement learning methods are able to capture this notion of deferred gratification. That is, they can learn to take actions, with short-term penalties, even, that can ultimately lead to large, long-term rewards or goal satisfaction. Reinforcement learning is a family of machine learning methods that are seeing great potential for emerging AI products.

Video 11: Summary – Machine Learning Methods (01:54)

This brings us to a good, intermediate point, to take stock of what we've learned so far. This week has focused on some general concepts of machine learning, including the difference between

supervised and unsupervised learning, and broadly, on the difference between, classification and regression.

Our goal this week, has been to build some basic intuition about a number of machine learning methods. The most fundamental of these are, basic linear classifiers, which we then extended to include logistic regression, and support vector machines, including the use of gradient descent, to train these classifiers. In contrast, we also looked at tree-based models, which have some advantages, such as being easy to interpret.

Finally, we also considered Bayesian methods, that emphasize a probability-based framework, where we can express and update our belief in the state of the world, as we gain, or gather, new data.

We will examine a broad family of important machine learning methods, neural networks, and especially architectures for deep learning. These include multi-layer perceptrons, or MLPs, for classification and regression. Convolutional neural networks, or CNNs, for image and video analysis, and recurrent neural networks, or RNNs, for time series and sequence modeling.

Together, deep learning and broader machine learning methods, will arm you with a solid introduction, to the large set of methods and models involved in AI products.